RAPPORT DE RÉALISATION

Sécurité du Cloud

Thème : Stockage objet évolutif avec Swift

Réalisé par	Encadrement
FOUDA DZOU AMBE Valdez KAMDOUM TEUFENG Josette	Mr. NGUIMBUS Emmanuel *Université Saint Jean* *Année Académique 2025-2026*

1. Introduction 3
2. Analyse de l'existant 3
3. Architecture de la solution 5
4. Choix technologiques et justification 6
5. Environnement de développement 7
6. Implémentation technique 9
7. Conclusion 12

1. Introduction

La gestion des données numériques connaît une croissance rapide, nécessitant des solutions de stockage capables d'évoluer facilement et de manière fiable. Dans ce contexte, le présent projet s'inscrit dans le cadre du module Sécurité du Cloud à l'Université Saint Jean et vise à développer une solution de stockage objet évolutive basée sur OpenStack Swift.

L'objectif est de concevoir un système capable d'assurer une synchronisation automatique et bidirectionnelle des données entre un environnement local Linux et une infrastructure de stockage distribué, à l'image des fonctionnalités offertes par des services comme OneDrive, mais adapté à l'écosystème Linux et aux contraintes techniques d'OpenStack.

Le présent rapport décrit l'ensemble du travail réalisé : l'analyse du contexte, l'architecture retenue, les choix technologiques effectués, l'environnement de développement mis en place, et les détails de l'implémentation technique de l'application SYNCORA v2.

2. Analyse de l'existant

2.1 Contexte et problématique

Aujourd'hui, les solutions de stockage cloud grand public telles que OneDrive, Google Drive ou Dropbox offrent une expérience utilisateur fluide avec synchronisation automatique, mais restent des solutions propriétaires fonctionnant principalement sur Windows ou avec des clients natifs limités sous Linux. Pour les environnements Linux en entreprise, il n'existe pas d'alternative libre et ouverte équivalente, combinant la puissance du stockage objet distribué et la simplicité d'un client de synchronisation transparent.

2.2 Limites des solutions existantes

OpenStack Swift est une plateforme de stockage objet distribué hautement disponible, scalable horizontalement et tolérante aux pannes. Cependant, dans son état brut, son interface reste purement technique :

- L'accès aux données se fait exclusivement via une API REST ou en ligne de commande (swift CLI), ce qui est inaccessible aux utilisateurs non-experts.
- Il n'existe pas de client natif Linux assurant une synchronisation bidirectionnelle automatique et transparente entre le système de fichiers local et les conteneurs Swift.
- L'interface Horizon (dashboard OpenStack) permet une gestion basique des objets mais n'offre pas de synchronisation en temps réel ni de montage de répertoire.

2.3 Besoins identifiés

L'analyse de l'existant a conduit à identifier les besoins suivants auxquels le projet répond :

Besoin identifié	Réponse apportée
Synchronisation automatique locale ↔ cloud	Détection des changements et upload/suppression automatiques
Surveillance en temps quasi réel des fichiers	Cycle de vérification toutes les 5 secondes
Gestion des conteneurs comme des dossiers	Mapping conteneur Swift ↔ dossier local dans ~/Documents/Syncora/
Déploiement en service système persistant	Intégration systemd avec démarrage automatique au boot
Authentification sécurisée via Keystone	Variables d'environnement OS_* injectées dans le service

3. Architecture de la solution

3.1 Vue d'ensemble

SYNCORA v2 est un démon système (daemon) écrit en langage C, qui s'exécute en arrière-plan sur une machine Linux. Il assure la synchronisation bidirectionnelle entre un répertoire local et les conteneurs d'un cluster OpenStack Swift. Le programme est découpé en modules indépendants, chacun ayant une responsabilité bien définie.

ARCHITECTURE GLOBALE DE SYNCORA v2

```
┌──────────────
```

Module	Fichiers	Responsabilité
Initialisation	init.c / init.h	Création de ~/Documents/Syncora/ et des sous-dossiers correspondant aux conteneurs Swift existants au démarrage.
Surveillance	monitor.c / monitor.h	Parcours de chaque dossier surveille, détection des créations, modifications et suppressions de fichiers par comparaison d'état.
Swift Client	swift.c / swift.h	Encapsulation des opérations Swift : upload d'objet, suppression d'objet, suppression de conteneur.
Orchestration	main.c	Boucle principale de synchronisation en 4 étapes, gestion dynamique des états des conteneurs, cycle de 5 secondes.

3.3 Cycle de synchronisation

À chaque itération de 5 secondes, la fonction refresh_sync_state() exécute les quatre étapes suivantes dans un ordre précis :

1. Suppression prioritaire (local → cloud) : si un dossier local a été supprimé, le conteneur Swift correspondant est immédiatement effacé.
2. Création locale (cloud → local) : tout nouveau conteneur détecté sur Swift qui n'existe pas localement est créé comme sous-dossier.
3. Création distante (local → cloud) : tout nouveau dossier créé localement déclenche la création du conteneur Swift correspondant via swift post.
4. Mise à jour de la liste de surveillance : la liste des FolderState est rafraîchie pour refléter l'état actuel.

4. Choix technologiques et justification

4.1 Langage C

Le développement en langage C a été retenu pour plusieurs raisons fondamentales. C'est le langage natif du système Linux, ce qui permet une intégration directe avec les API POSIX du noyau (stat, opendir, readdir, mkdir) sans surcoût d'abstraction. Les programmes C compilés sont également légers en mémoire et en CPU, qualité essentielle pour un démon qui tourne en permanence en arrière-plan. Enfin, C offre un contrôle précis de la gestion des ressources système.

4.2 OpenStack Swift

OpenStack Swift a été choisi comme backend de stockage objet pour ses capacités natives de stockage distribué, de tolérance aux pannes par réplication sur plusieurs nœuds, et de scalabilité horizontale. L'accès via la swift CLI (python3-swiftclient) disponible sous Linux permet

d'interagir avec l'API REST Swift sans développer un client HTTP complet, ce qui simplifie l'implémentation tout en restant pleinement fonctionnel.

4.3 OpenStack Keystone

L'authentification est déléguée à OpenStack Keystone, le service d'identité standard d'OpenStack. Les credentials sont fournis via des variables d'environnement standardisées (OS_AUTH_URL, OS_USERNAME, OS_PASSWORD, OS_PROJECT_NAME, OS_IDENTITY_API_VERSION), injectées directement dans le service systemd. Cette approche évite de stocker des mots de passe dans le code source et respecte les bonnes pratiques de gestion des secrets en environnement Linux.

4.4 systemd

Le déploiement via systemd permet d'intégrer SYNCORA v2 comme un service système de première classe. L'unité de service configurée assure le démarrage automatique après le réseau (After=network-online.target), le redémarrage automatique en cas de crash (Restart=always, RestartSec=10), et la journalisation native via journalctl, sans développer de mécanisme de logging personnalisé.

Technologie	Rôle dans le projet	Justification
C (GCC)	Langage principal du démon	Légèreté, API POSIX native, performance
OpenStack Swift	Backend de stockage objet	Scalabilité, distribution, haute disponibilité
swift CLI (swiftclient)	Interface avec l'API REST Swift	Disponible nativement sous Linux, fiable
OpenStack Keystone	Authentification et autorisation	Standard OpenStack, gestion des tokens
systemd	Gestion du service et des logs	Intégration OS, auto-restart, journalisation
GCC + Makefile	Compilation et build	Standard de facto pour projets C sous Linux
Git / GitHub	Gestion de versions	Collaboration et traçabilité du code

5. Environnement de développement

5.1 Prérequis système

Le développement et le déploiement de SYNCORA v2 nécessitent un environnement Linux (Ubuntu recommandé) avec les composants suivants :

- Système d'exploitation : Linux (Ubuntu 20.04 ou supérieur)
- Compilateur : GCC avec support des flags -Wall
- Client OpenStack : python3-swiftclient et python3-openstackclient
- Système d'init : systemd (présent par défaut sur Ubuntu)
- Accès réseau : connectivité vers le controller OpenStack (port 5000 pour Keystone)

5.2 Installation des dépendances

Les dépendances nécessaires s'installent via les commandes suivantes :

```
sudo apt update
sudo apt install python3-swiftclient python3-openstackclient
```

5.3 Configuration réseau

L'accès au controller OpenStack requiert la résolution du nom d'hôte controller. Cette configuration est réalisée en ajoutant une entrée dans le fichier /etc/hosts :

```
IP_ADDRESS  controller    # Adresse IP du controller OpenStack
```

5.4 Configuration des accès OpenStack

Un fichier de configuration des variables d'environnement OpenStack est fourni dans le dossier config/ du dépôt. Il contient l'ensemble des paramètres nécessaires à l'authentification Keystone :

```
export OS_PROJECT_DOMAIN_NAME=Default
export OS_USER_DOMAIN_NAME=Default
export OS_PROJECT_NAME=myproject
export OS_USERNAME=myuser
export OS_AUTH_URL=http://<controller>:5000/v3
export OS_IDENTITY_API_VERSION=3
export OS_IMAGE_API_VERSION=2
```

5.5 Outils de développement

Outil	Version	Usage
GCC	11.x (Ubuntu 22.04)	Compilation du projet C
Make	4.3+	Automatisation du build
python3-swiftclient	4.x	CLI Swift pour upload/delete
systemd	249+	Gestion du service syncora
Git	2.x	Versioning du code source
visual studio code	1.x	editeur principale pour l'ecriture et la navigation dans le code C
journalctl	(systemd)	Consultation des logs en temps réel

6. Implémentation technique

6.1 Compilation du projet

Le projet se compile via un Makefile minimal. Les quatre fichiers sources C sont compilés ensemble pour produire le binaire syncora :

```
# Compilation
make clean && make

# Déploiement du binaire
sudo cp syncora /usr/local/bin/syncora
```

Le Makefile utilisé est le suivant :

```
CC=gcc
CFLAGS=-Wall

all:
    gcc main.c init.c monitor.c swift.c -o syncora -Wall

clean:
    rm -f syncora
```

6.2 Module d'initialisation — init.c

Au démarrage, la fonction setup_containers() réalise l'initialisation complète de l'espace de travail. Elle crée le répertoire racine ~/Documents/Syncora/ si celui-ci n'existe pas encore, puis interroge le cluster Swift via swift list pour récupérer la liste des conteneurs existants. Pour chaque

conteneur distant détecté, un sous-dossier local correspondant est automatiquement créé, assurant ainsi que l'état local reflète l'état distant dès le lancement du service.

```
// Extrait de init.c
void setup_containers() {
    char *home = getenv("HOME");
    snprintf(base_path, sizeof(base_path), "%s/Documents/Syncora", home);
    mkdir(base_path, 0755);

    fp = popen("swift list", "r");
    while (fgets(container_name, sizeof(container_name), fp)) {
        // Crée le dossier local pour chaque conteneur distant
        mkdir(full_folder_path, 0755);
    }
}
```

6.3 Module de surveillance — monitor.c

La fonction monitor_directory() parcourt un dossier surveillé et détecte trois types d'événements en comparant l'état courant du répertoire avec l'état mémorisé lors du cycle précédent (stocké dans la structure FolderState) :

- **Création de fichier : un fichier présent dans le scan courant mais absent de l'état précédent déclenche un upload vers Swift.**
- **Modification de fichier : un fichier présent dans les deux états mais avec un timestamp st_mtime différent déclenche un re-upload.**
- **Suppression de fichier : un fichier présent dans l'état précédent mais absent du scan courant déclenche une suppression sur Swift.**

```
// Détection modification (extrait monitor.c)
if (current_files[i].last_modified != state->previous_files[j].last_modified) {
    printf("[MODIF] %s\n", current_files[i].name);
    swift_upload(container_name, full_file_path);
}
```

Les structures de données utilisées permettent de surveiller jusqu'à 1024 fichiers (MAX_FILES) par conteneur et jusqu'à 100 conteneurs (MAX_CONTAINERS) simultanément.

6.4 Module Swift — swift.c

Ce module encapsule les trois opérations d'interaction avec le cluster Swift. Chaque opération est réalisée via un appel à la swift CLI, ce qui exploite l'authentification Keystone déjà configurée dans les variables d'environnement du service :

```
// Upload d'un fichier (--changed évite les uploads inutiles)
swift upload <container> '<filepath>' --object-name '<filename>' --changed

// Suppression d'un objet
swift delete <container> '<filename>'

// Suppression d'un conteneur entier
swift delete <container>
```

Le flag --changed dans la commande d'upload est particulièrement important : il compare le MD5 de l'objet local avec celui stocké sur Swift avant d'effectuer le transfert, évitant ainsi des uploads redondants pour des fichiers identiques.

6.5 Boucle principale — main.c

La fonction main() orchestre l'ensemble du processus. Après l'initialisation via setup_containers(), elle entre dans une boucle infinie qui, toutes les 5 secondes, exécute refresh_sync_state() pour maintenir la cohérence entre l'état local et l'état distant, puis appelle monitor_directory() pour chaque conteneur surveillé.

```
// Boucle principale (extrait main.c)
while (1) {
    refresh_sync_state(states, &count, home);  // Sync état conteneurs
    for (int i = 0; i < count; i++) {
        monitor_directory(&states[i]);          // Surveillance fichiers
    }
    sleep(5);                                   // Cycle 5 secondes
}
```

6.6 Déploiement en service systemd

SYNCORA v2 s'installe comme un service systemd persistant. Le fichier de service syncora.service, fourni dans le dossier config/ du dépôt, configure l'exécution du démon avec toutes les variables OpenStack nécessaires et assure son redémarrage automatique en cas d'échec :

```
[Unit]
Description=Service de Synchronisation Syncora Cloud (OpenStack)
After=network-online.target

[Service]
Type=simple
User=username
Environment="OS_AUTH_URL=http://controller:5000/v3"
Environment="OS_PROJECT_NAME=nom_du_projet"
Environment="OS_USERNAME=utilisateur"
```

```
Environment="OS_PASSWORD=mot_de_passe"
ExecStart=/usr/local/bin/syncora
Restart=always
RestartSec=10

[Install]
WantedBy=multi-user.target
```

Les commandes d'activation et de démarrage du service sont les suivantes :

```
sudo systemctl daemon-reload
sudo systemctl enable syncora.service
sudo systemctl start syncora.service
```

6.7 Logs et supervision

Le service s'intègre nativement au système de journalisation de systemd. Deux commandes permettent le suivi en temps réel :

```
# Suivi des logs en temps réel
journalctl -u syncora.service -f

# Vérification du statut du service
systemctl status syncora.service
```

Les événements enregistrés incluent les opérations de synchronisation ([NEW], [MODIF], [DEL], [CLOUD], [SYNC-IN]) permettant un diagnostic précis de l'activité du démon.

6.8 Utilisation de l'application

Une fois le service démarré, l'utilisation est entièrement transparente pour l'utilisateur. L'application crée automatiquement le dossier ~/Documents/Syncora/ et y génère un sous-dossier pour chaque conteneur associé au compte Swift configuré. Tout fichier ajouté, modifié ou supprimé dans ces dossiers est synchronisé automatiquement avec le cluster Swift dans les 5 secondes suivant l'opération.

Fonctionnement utilisateur

```
~/Documents/Syncora/
├── mon_projet/      ← Dossier synchronisé avec le conteneur 'mon_projet' sur Swift
│   ├── rapport.pdf  ← Tout ajout ici → uploadé sur Swift automatiquement
```

```
│   └── data.csv      ← Toute modification ici → mis à jour sur Swift
└── backup/           ← Tout nouveau dossier créé ici → conteneur créé sur Swift
```

7. Conclusion

Le projet SYNCORA v2 répond concrètement au besoin de synchronisation automatique et bidirectionnelle entre un système de fichiers Linux et un cluster OpenStack Swift. L'application développée en langage C s'intègre nativement à l'écosystème Linux via systemd, offre une surveillance continue des répertoires et gère de manière autonome les créations, modifications et suppressions de fichiers et de conteneurs.

L'architecture modulaire retenue — séparant l'initialisation, la surveillance, les opérations Swift et l'orchestration — rend le code lisible, maintenable et extensible. Le déploiement en service système garantit la persistance et la résilience du processus de synchronisation, tandis que l'intégration Keystone assure une authentification conforme aux standards OpenStack.

Ce travail constitue une base technique solide et fonctionnelle pour un client de synchronisation cloud libre sous Linux, démontrant la faisabilité d'une alternative open source aux clients propriétaires dans un environnement de stockage objet distribué.